Tutorial 3

- Tutorial 3: https://docs.google.com/presentation/d/1GjGW39ufWI3bDcy7rMcaRYin_11soNIzCh2CtzO7580/edit?slide=id.p#slide=id.p
- Written by Milad Abdi and some AI assistance.

Transformations in ROS

Goal:

- Maintain relationships between reference frames over time
- Transform points / poses between coordinate frames
- Broadcast this information across ROS so any node can subscribe and perform transformations

Packages (ROS 2)

- `tf2`: Core math library for handling the transformations
- `tf2_ros`: ROS wrapper around `tf2`
- `tf2_tools`: Utilities for debugging
  Official tutorial: https://docs.ros.org/en/humble/Tutorials/Tf2/Tf2-Main.html
  ROS conventions (units, orientation, rotation, covariance): https://www.ros.org/reps/rep-0103.html
  Standard mobile base frame names: https://www.ros.org/reps/rep-0105.html

TF Tree

- The TF is a **directed tree of coordinate frames**. The tree contains:
  - All frames
  - Relative transforms
  - Who is broadcasting each transform
  - Broadcast frequency
- If a frame is connected to the TF tree, we can obtain transforms between it and any other connected frame.
- Visualization is extremely useful for debugging.
  Monitor TF tree:

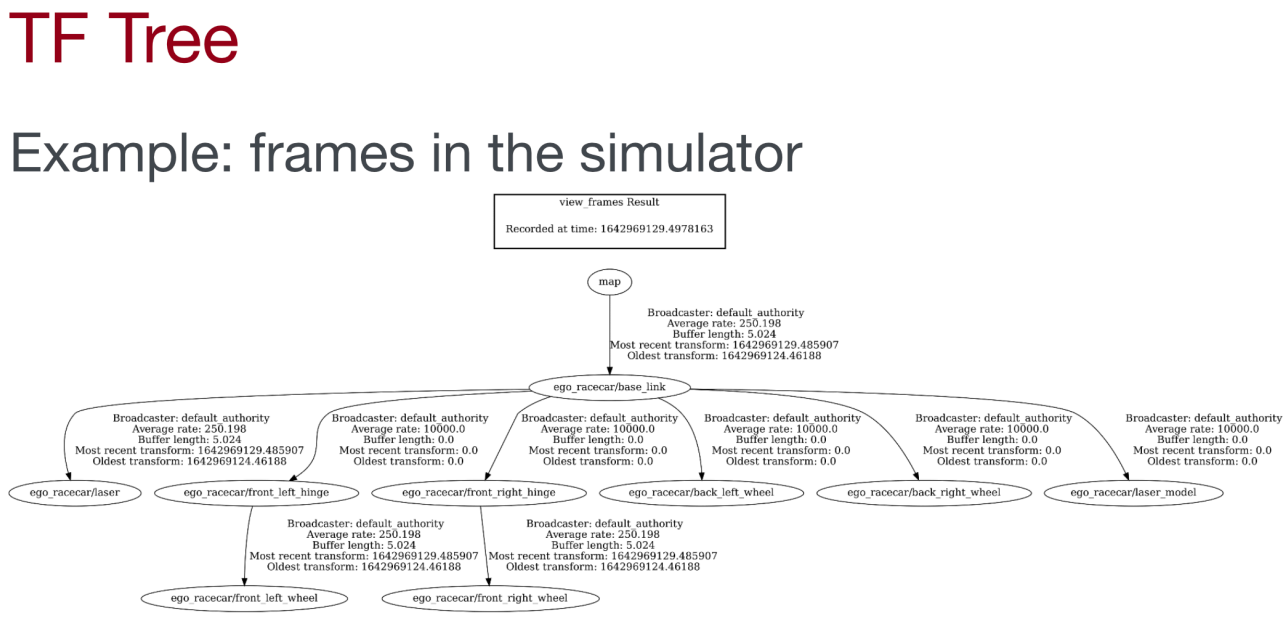
```
ros2 run tf2_ros tf2_monitor
ros2 run tf2_ros tf2_echo <src_frame> <target_frame> [echo_rate]
```

Visualize TF tree:

```
ros2 run tf2_tools view_frames.py
```

Creates:

- `frames.gv`
- `frames.pdf`
  Example: Frames in the simulator



Static Transforms

Used when:

- Frames do not move relative to each other
- Example: `laser` ↔ `base_link`
  Typically defined in:
- URDF / robot description files
  URDF tutorial: https://docs.ros.org/en/humble/Tutorials/URDF/URDF-Main.html

Broadcasting Static Transform (CLI)

Using RPY:

```
ros2 run tf2_ros static_transform_publisher x y z yaw pitch roll frame_id
child_frame_id
```

Using quaternions:

```
ros2 run tf2_ros static_transform_publisher x y z qx qy qz qw frame_id
child_frame_id
```

Broadcasting Static Transform (Launch File)

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
   return LaunchDescription([
      Node(
         package='tf2_ros',
         executable='static_transform_publisher',
         arguments=['0', '0', '1', '0', '0', '0', 'world', 'my_frame'],
      ),
   ])
```

TF2 API Concepts

- **Broadcaster**:
  - Sends a `TransformStamped` message
  - Adds transform to TF tree
  - Specifically:
    - Creates a `TransformStamped` message
    - Fills in:
      - Parent frame
      - Child frame
      - Translation
      - Rotation
      - Timestamp
    - Sends it to the TF system
  - Conceptually: “At time t, frame B is located here relative to frame A.”
  - If it’s static -> publish once
  - If it’s dynamic (like odometry) -> publish repeatedly
- **Listener**:
  - Listens to the TF tree and finds transformation with given frames and time
  - Specifically:
    - The listener:
      - Subscribes to the TF tree

        - Stores all transforms internally
        - Computes requested transforms on demand
    - When you call: `lookupTransform(toFrame, fromFrame, time)`:
        1. Finds path in the tree
        2. Multiplies transforms along the path
        3. Returns final transform
- **Buffer**:
    - Stores transform history over time
    - Allows querying transforms at specific timestamps
- **Coordinate Transformations**:
    - Transform a geometric message from one frame to another

TF2 API – Transform Broadcaster (C++)

- In ROS 2, a `tf2_ros.TransformBroadcaster` object can be used in a node to broadcast up-to-date transformation matrices, or quaternions between frames.

```
Tf_publisher_ = std::make_shared<tf2_ros::StaticTransformBroadcaster>(this);
    geometry_msgs::msg::TransformStamped t;
    …
    tf_publisher_->sendTransform(t);
```

Tutorial: https://docs.ros.org/en/humble/Tutorials/Tf2/Writing-A-Tf2-Static-Broadcaster-Cpp.html

TF2 API – Transform Listener (C++)

```
tf_buffer_ = std::make_unique<tf2_ros::Buffer>(this->get_clock());
transform_listener_ = std::make_shared<tf2_ros::TransformListener>
(*tf_buffer_);
transformStamped = tf_buffer_->lookupTransform(
                                    toFrameRel,
                                    fromFrameRel,
                                    tf2::TimePointZero);
```

Tutorial: https://docs.ros.org/en/humble/Tutorials/Tf2/Writing-A-Tf2-Listener-Cpp.html

Performing Transformations in ROS 2 (C++)

- Use `tf2_ros::BufferInterface::transform()` to transform different data types. See https://docs.ros2.org/foxy/api/tf2_ros/classtf2__ros_1_1BufferInterface.html#a1aa26a4c7b240c2c6207fa5184e0111c for the (foxy) API.

- 6 total different function signatures.
- **Recap**:
  1. Every sensor measures in its own frame.
  2. Planning and mapping happen in a global frame.
  3. TF lets you convert between them.
  4. TF is just organized rigid body transformations with timestamps.
  5. The TF tree must remain connected.